Universidad Nacional Autónoma de México

Computer Engineering

Compilers

Lexer

Students:
320198687 320301238 321132927 321132477 321332086

Gruop: 5
Semester:
2026-II

México,CDMX. March 2026

Contents

1 Introduction **2**

2 Theoretical Framework **2**

3 Development **3**

4 Results **4**

5 Conclusions **11**

6 Bibliography **11**

1 Introduction

The present project corresponds to the development of a lexical analyzer. Lexical analysis is the first phase of the compilation process, also known as a lexer, it is responsible for reading the source code and grouping characters into meaningful units called tokens, such as reserved words, identifiers, operators and constants.
The core objective of this project is to operationalize the theoretical foundations of lexical analysis within a practical compiler framework. By leveraging principles from Formal Languages and Automata—specifically Regular Expressions for pattern recognition and Deterministic Finite Automata (DFA) abstractions—this implementation demonstrates the conversion of a source code character stream into a structured sequence of tokens. This process underscores the critical role of the lexer as the initial phase in ensuring the efficiency and correctness of the subsequent stages of a compiler. Ultimately, the project delivers a functional lexical analyzer capable of parsing high-level code and categorizing its constituents into tokens.
In this project a lexical analyzer was implemented in Python that traverses the source code character by character, the system allows reading the code in two ways: entering it manually or loading a text file, once processed it classifies each element found into categories.
The lexer traverses the source code character by character to identify and classify lexical units (tokens). The system is capable of recognizing more than 40 tokens distributed in categories such as reserved words, identifiers, operators, punctuation, constants, string literals, character literals and comments.
The analyzer detects lexical errors such as unclosed strings or unknown characters. As a theoretical complement, a Context-Free Grammar with left factoring is presented representing one of the input examples used.

2 Theoretical Framework

For the development two approaches were evaluated: the first was to use exclusively Python's re library with regular expressions, the second which was the one we chose consists of traversing the source code character by character applying explicit conditions for each type of token, handling cases such as multi-character operators, single-line comments, multi-line comments, Python-style directives, string literals, character literals and lexical errors such as unclosed strings. Although the re module is used specifically to recognize numeric constants the core of the analyzer is built on manual traversal directly connecting the implementation with the theoretical concepts of lexical analysis.

This project implements a Deterministic Finite Automata (DFA) model through a character-by-character scanning process. The DFA transitions are operationalized using Regular Expressions (RegEx) and Python's re library, allowing the system to navigate through various states until reaching an acceptance state, where a token is generated.

The Symbol Table is managed implicitly during this phase by capturing and stor-

ing the unique names of identifiers found in the source code. Before registering a new identifier, the system performs an existence check to ensure uniqueness within the current scope. Finally, the implementation adheres to the 'Maximal Munch' (longest match) principle to resolve potential lexical ambiguities. This strategy is essential when processing operators of varying lengths, as it prevents the scanner from incorrectly decomposing a multi-character operator into a sequence of smaller, independent operators.

3 Development

The program implements a modular design by its structural distribution, segregating the lexer's responsibilities into distinct functional components.

- **Token Dictionary**. This component serves as a dictionary that categorizes lexemes into predefined terminal sets, distinguishing between fixed strings—such as keywords (int, return), operators, and punctuation—and dynamic patterns. It employs Regular Expressions (RegEx) to identify identifiers, numeric constants, and multi-line comments. Additionally, it includes specific logic to handle non-token elements like whitespace and unknown tokens, ensuring the lexer can detect and report invalid characters without halting the process.

- **Lexical Analysis Engine (tokenizar)**. Represent the most important and critical function in the entire program because processes the source code as a stream of characters, systematically identifying and extracting lexemes to transform them into categorized tokens.
  During the development finite state machines were used for lexical analyzers, it is directly reflected in the main loop while posicion $<$ longitud of the lexer, in each iteration it reads a character from the source code and decides which token is starting.
  Each if/elif block represents a state transition allowing the lexer to correctly recognize multi-character operators, string literals, character literals, comments and lexical errors such as unclosed strings.

  State Machines

  On the other hand, the variable resultado works as a basic Symbol Table. By using a Python set, the program automatically removes repeated identifiers because sets do not allow duplicate elements. This way, we save only the unique names found in the code for the final report.
  Also, the Maximal Munch principle is applied in the if-elif structure where we handle operators. We sort the operators by their length from longest to shortest. This ensures the program compares the longest patterns first, avoiding mistakes like identifying two separate + signs when there is actually a ++ operator.

Table 1: State Transition Table

Current State	**Character Read**	**Next State**	**Token Recognized**
Start	`/`	S1	—
S1	`/`	Line Comment	`comment`
S1	`*`	Block Comment	`comment`
S1	other	Start	`/` operator
Start	`=`	S2	—
S2	`=`	Start	`==` operator
S2	other	Start	`=` operator
Start	`"`	S3	—
S3	`"`	Start	`literal_string`
S3	end of line without `"`	Error	`unknown`
Start	letter or `_`	S4	—
S4	letter, digit or `_`	S4	—
S4	other	Start	`keyword` or `identifier`
Start	digit	S5	—
S5	digit or `.`	S5	—
S5	other	Start	`constant`
Start	`$`, `@`, ¿	Start	`unknown`

- **Data visualization**. The data is visualized directly in the terminal. First, the program displays the processed source code. Then, it shows the generated tokens organized by their classification, along with the total count of tokens identified during the process. It also displays the comments found in the code and the text contained in `literal_strings`.

Table 2: Token Classification and Lexeme Examples

Token Category	**Lexeme Examples**
Keyword	int, return, if
Identifier	main, x, _variable
Punctuation	(,), {, }, ;, ,
Operator	=, +, *, ++
Constant	2, 3, 4
Literal String	"Hola Mundo"

4 Results

The following screenshots present the final results oh the lexer execution using the two different input methods.

Manual input — function with variables

The following output corresponds to manually entering a basic C function containing variable declarations, arithmetic operations and a return statement.

```
PS C:\Users\maria> & C:/Users/maria/AppData/Local/Microsoft/Win
=== L E X E R ===
Opciones:
1. Ingresar código manualmente
2. Leer desde archivo
Selecciona una opción (1/2): 1
Ingresa tu código (presiona enter dos veces para terminar):
int main() {
    int x = 10;
    int y = 20;
    int suma = x + y;
    return 0;
}



==================================================
Código
int main() {
    int x = 10;
    int y = 20;
    int suma = x + y;
    return 0;
}


==================================================

==================================================
TOKENS ENCONTRADOS
==================================================

KEYWORD: return, int

IDENTIFIER: main, y, x, suma

PUNCTUATION: }, ;, {, ), (

OPERATOR: +, =

CONSTANT: 0, 10, 20

==================================================
COUNT:
Total de tokens encontrados: 26
Comentarios: 0
```

The lexer correctly identified 26 tokens distributed across 5 categories: 2 keywords (`int`, `return`), 4 identifiers (`main`, `x`, `y`, `suma`), 5 punctuation symbols, 2 operators (`+`, `=`) and 3 constants (0, 10, 20). No comments or lexical errors were detected.

Manual input - complex operators and comments

The following output corresponds to manually entering code containing compound operators, logical expressions and single-line comments.

```
PS C:\Users\maria\AppData\Local\Temp\24fc26c0-0c9e-4bfb-b10c-e6a0128f47b2_tokenizador.zip.7b2\tokenizador>  & 'c
aria\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61910' '--' 'C:\Us
.zip.7b2\tokenizador\lexer.py'
=== L E X E R ===
Opciones:
1. Ingresar código manualmente
2. Leer desde archivo
Selecciona una opción (1/2): 1
Ingresa tu código (presiona enter dos veces para terminar):
x = a+++b; // Debería ser: id(a), op(++), op(+), id(b)
y = (a>=b)&&(c!=d);
z = x<<=2; // Operador compuesto de asignación con desplazamiento



==================================================
Código
x = a+++b; // Debería ser: id(a), op(++), op(+), id(b)
y = (a>=b)&&(c!=d);
z = x<<=2; // Operador compuesto de asignación con desplazamiento


==================================================

==================================================
TOKENS ENCONTRADOS
==================================================

IDENTIFIER: a, x, b, y, c, d, z

PUNCTUATION: ), ;, (

OPERATOR: !=, ++, +, >=, <<, &&, =

CONSTANT: 2

COMMENT: // Debería ser: id(a), op(++), op(+), id(b), // Operador compuesto de asignación con desplazamiento

==================================================
COUNT:
Total de tokens encontrados: 30
Comentarios: 2
```

The lexer correctly identified 30 tokens distributed across 5 categories: 7 identifiers (`a`, `b`, `c`, `d`, `x`, `y`, `z`), 3 punctuation symbols, 7 operators (`!=`, `++`, `+`, `>=`, «, `&&`, `=`), 1 constant and 2 comments. No lexical errors were detected.

Manual input -comments and directives

The following output corresponds to manually entering a small code containing single-line comments, multi-line comments and a directive.

```
=== L E X E R ===
Opciones:
1. Ingresar código manualmente
2. Leer desde archivo
Selecciona una opción (1/2): 1
Ingresa tu código (presiona enter dos veces para terminar):
int/*comentario*/x=10;
// Comentario de una línea que termina el código
/* Comentario multilínea
   con salto de línea */ printf("Hola"); # Directiva tipo Python



==================================================
Código
int/*comentario*/x=10;
// Comentario de una línea que termina el código
/* Comentario multilínea
   con salto de línea */ printf("Hola"); # Directiva tipo Python


==================================================

==================================================
TOKENS ENCONTRADOS
==================================================

KEYWORD: printf, int

IDENTIFIER: x

PUNCTUATION: (, ), ;

OPERATOR: =

CONSTANT: 10

LITERAL_STRING: "Hola"

COMMENT: /* Comentario multilínea
   con salto de línea */, /*comentario*/, // Comentario de una línea que termina el código, # Directiva tipo Python

==================================================
COUNT:
Total de tokens encontrados: 14
Comentarios: 4
```

The lexer identified 14 tokens distributed across 7 categories: 2 keywords (`printf`, `int`), 1 identifier (`x`), 3 punctuation symbols, 1 operator (`=`), 1 constant (10), 1 string literal (`"Hola"`) and 4 comments, including multi-line comments, single-line comments and the directive, no lexical errors were detected.

Reading from file - lexical errors

The following output corresponds to reading the file code8.txt, which contains unrecognized characters in C Language.

```
=== L E X E R ===
Opciones:
1. Ingresar código manualmente
2. Leer desde archivo
Selecciona una opción (1/2): 2
Sugerencia: code1.txt, code2.txt, code3.txt
Nombre del archivo a leer: code8.txt

==================================================
Código
int valor = 100$;
float @temperatura = 36.5;
¿Por_que_esto_no_falla?;

==================================================

==================================================
TOKENS ENCONTRADOS
==================================================

KEYWORD: int

IDENTIFIER: temperatura, valor, Por_que_esto_no_falla, float

PUNCTUATION: ;

OPERATOR: =

CONSTANT: 100, 36.5

UNKNOWN: ?, ¿, $, @

==================================================
COUNT:
Total de tokens encontrados: 16
Comentarios: 0
```

The lexer identified 16 tokens distributed across 6 categories: 1 keyword (`int`), 4 identifiers (`temperatura`, `valor`, `Por_que_esto_no_falla`, `float`), 1 punctuation symbol, 1 operator (=), 2 constants (100, 36.5) and 4 unknown tokens (`?`, `¿`, `$`, `@`), which represent lexical errors correctly detected by the analyzer.

Reading from file - Special identifiers

The following output corresponds to reading the file code9.txt which contains identifiers such as underscores, uppercase letters and numbers.

```
=== L E X E R ===
Opciones:
1. Ingresar código manualmente
2. Leer desde archivo
Selecciona una opción (1/2): 2
Sugerencia: code1.txt, code2.txt, code3.txt
Nombre del archivo a leer: code9.txt

==================================================
Código
int _variable_oculta = 1;
int VariableConNumeros123 = 2;
int a_b_c_d_e = 3;

==================================================

==================================================
TOKENS ENCONTRADOS
==================================================

KEYWORD: int

IDENTIFIER: _variable_oculta, VariableConNumeros123, a_b_c_d_e

PUNCTUATION: ;

OPERATOR: =

CONSTANT: 2, 1, 3

==================================================
COUNT:
Total de tokens encontrados: 15
Comentarios: 0
```

The lexer correctly identified 15 tokens distributed across 5 categories: 1 keyword (`int`), 3 identifiers (`_variable_oculta`, `VariableConNumeros123`, `a_b_c_d_e`), 1 punctuation symbol, 1 operator (=) and 3 constants (1, 2, 3). No comments or lexical errors were detected, this demonstrates that the analyzer can correctly recognize identifiers with different valid formats.

Reading from file - unclosed string

The following output corresponds to reading the file code10.txt which contains a string literal without a closing quotation mark.

```
=== L E X E R ===
Opciones:
1. Ingresar código manualmente
2. Leer desde archivo
Selecciona una opción (1/2): 2
Sugerencia: code1.txt, code2.txt, code3.txt
Nombre del archivo a leer: code10.txt

==================================================
Código
printf("Cadena sin cierre de comillas);
int y = 20;

==================================================

==================================================
Error léxico
==================================================

KEYWORD: printf, int

IDENTIFIER: comillas, y, cierre, sin, Cadena, de

PUNCTUATION: ), ;, (

OPERATOR: =

CONSTANT: 20

UNKNOWN: "

==================================================
COUNT:
Total de tokens encontrados: 15
Comentarios: 0
Errores léxicos
"
```

The lexer identified 15 tokens distributed across 6 categories: 2 keywords (`printf`, `int`), 7 identifiers (`comillas`, `y`, `cierre`, `sin`, `Cadena`, `de`), 3 punctuation symbols, 1 operator (=), 1 constant (20) and 1 unknown token ("), which represents the lexical error generated by the unclosed string. The analyzer correctly detected and reported the lexical error without halting execution.

5 Conclusions

The implementation of a lexical analyzer from scratch demonstrated that the theoretical model of Deterministic Finite Automata can be directly translated into code through a character-by-character traversal where each `if/elif` block represents a state transition.
The effectiveness of Regular Expressions (RegEx) was a decisive factor in the development of this lexer. They acted as the formal bridge between the mathematical theory of languages and the Python implementation. By using RegEx, the system could efficiently recognize dynamic patterns, such as identifiers and numeric constants, which would be extremely difficult to categorize using only simple string comparisons
The Maximal Munch principle proved fundamental to resolve lexical ambiguities particularly in the recognition of multi-character operators such as `++`, `>=` or `<<`. Without this part our analyzer would not be able to decompose these operators into smaller independent tokens.
The test cases managed to confirm that our analyzer correctly handles cases such as unclosed strings, unknown characters, directives and multi-line comments and classifies them appropriately without halting execution, concluding that a lexer must not only recognize valid tokens but also detect and report invalid tokens.

6 Bibliography

[1] Universidad Nacional Autónoma de México, (s.f.), *Apuntes de Compiladores*, Ptolomeo UNAM. Available: `http://www.ptolomeo.unam.mx:8080/jspui/bitstream/132.248.52.100/10230/1/7J%20APUNTES%20DE%20COMPILADORES_OCR.pdf`. [Accessed: Mar. 14, 2026].

[2] Bruni, C. (2019), *Maximal Munch and Context Free Grammars*, CS 241 Lecture 9, University of Waterloo. Available: `https://cs.uwaterloo.ca/~cbruni/CS241Resources/lectures/2019_Winter/CS241L09_cfg_post.pdf`. [Accessed: Mar. 15, 2026].